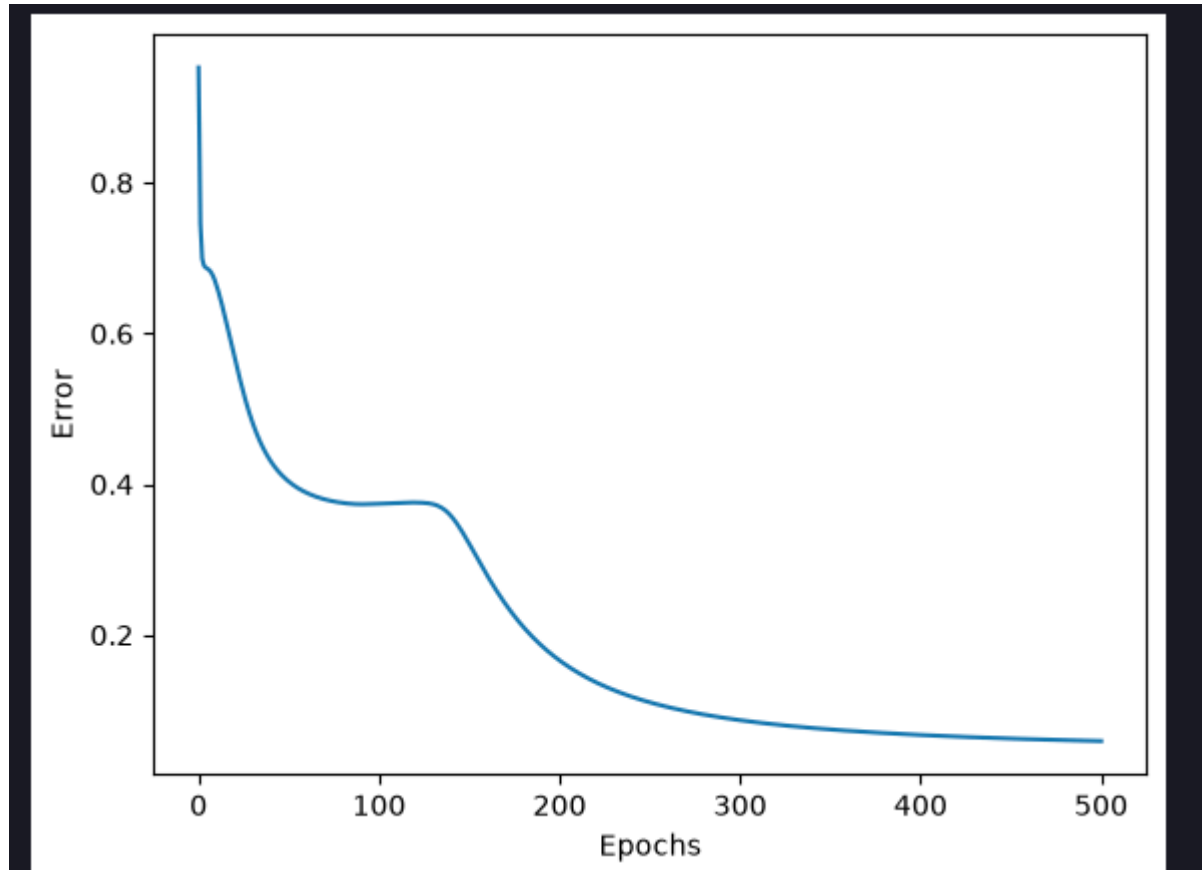


Perceptron: Ejercicio 01:

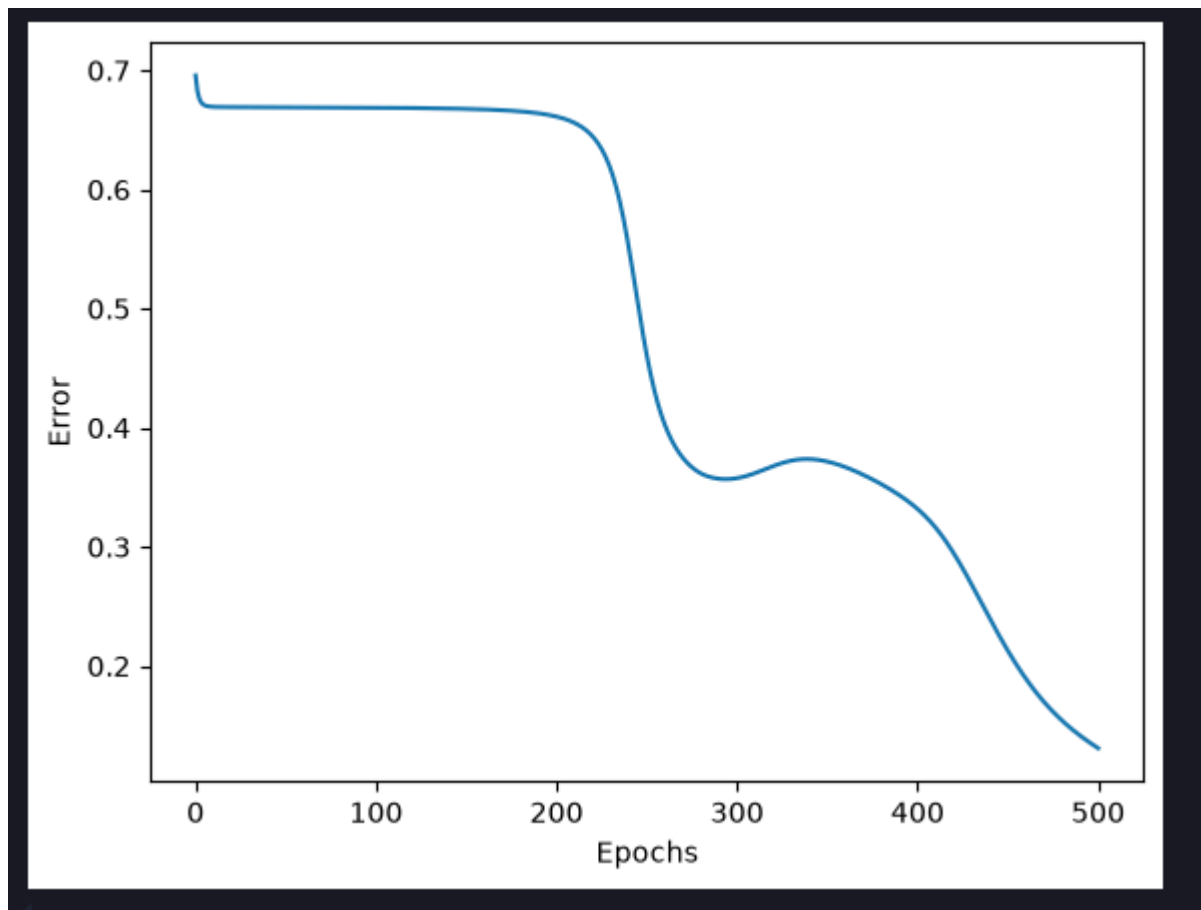
Implementación a mano con tres capas :

Graphico de error:



Implementación a mano con dos capas más:

Graphico de error:



Keras original :

`model.summary()` :

Model: "sequential"

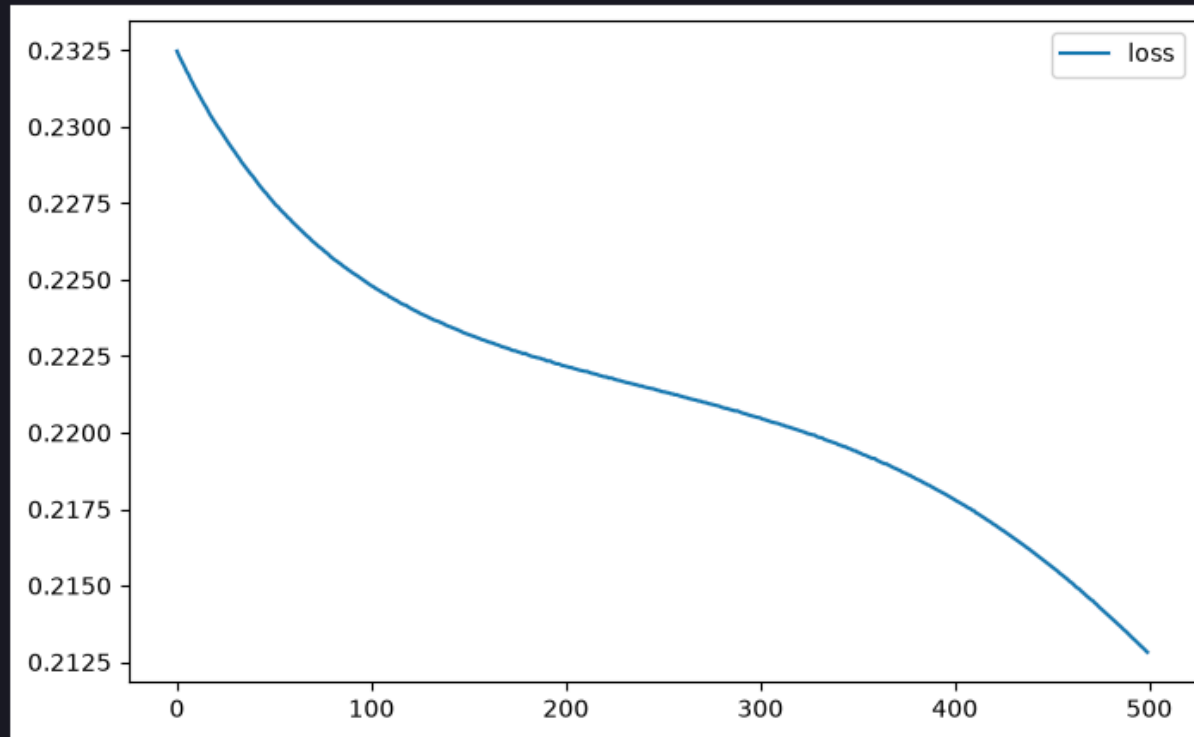
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 27 (108.00 B)

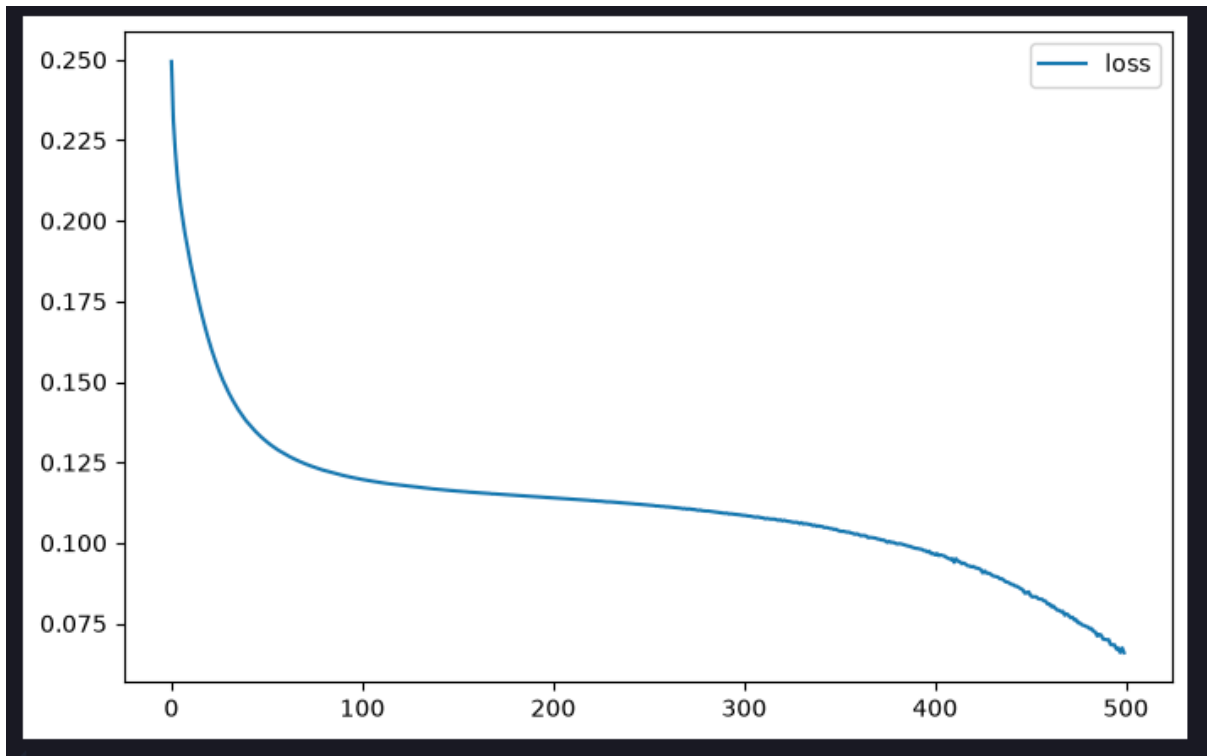
Trainable params: 27 (108.00 B)

Non-trainable params: 0 (0.00 B)

Loss graphico con batch size = 32:



Loss graphico con batch size = 1:



Keras con dos capas más :

Model.summary() :

Model: "sequential_1"

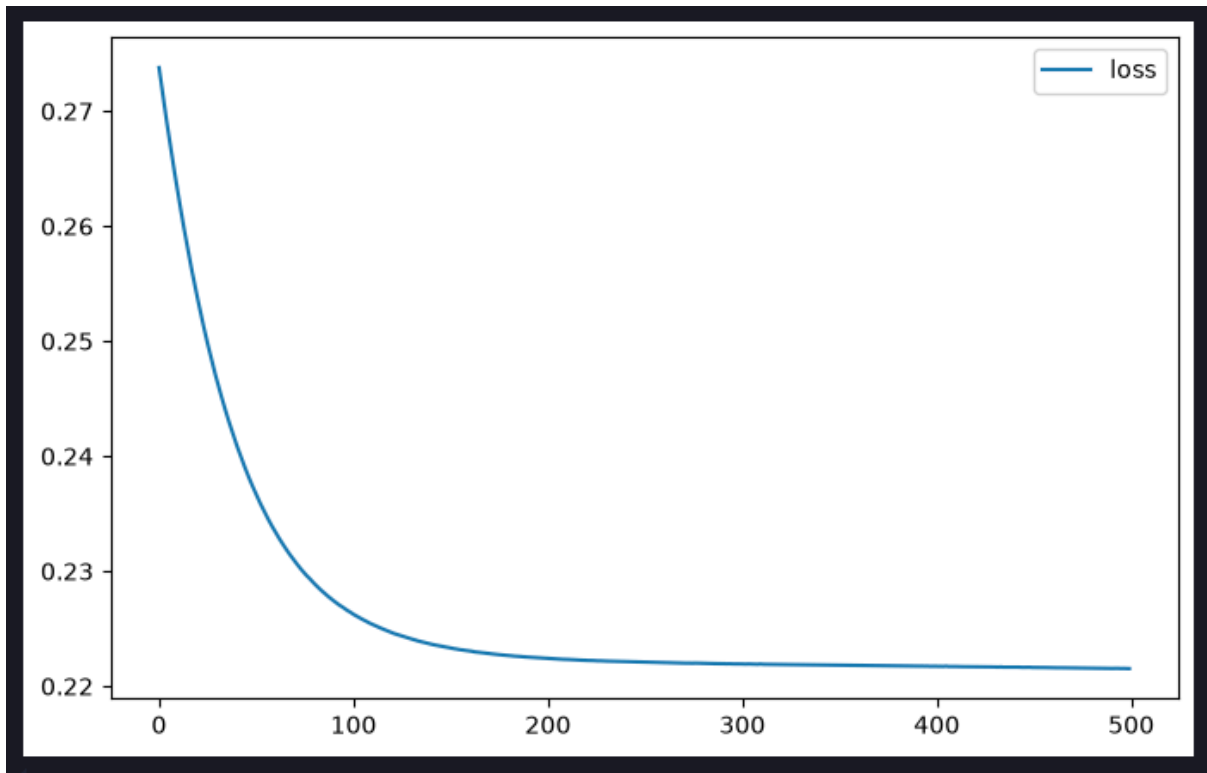
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)

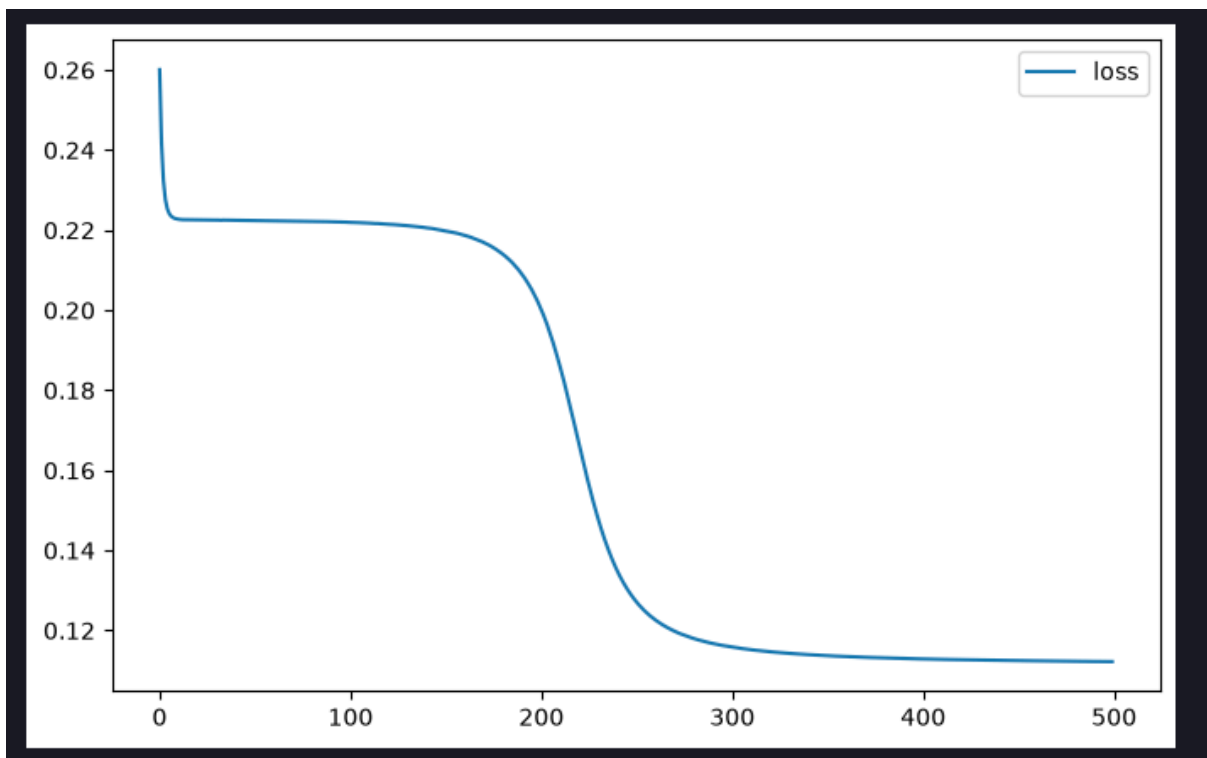
Trainable params: 51 (204.00 B)

Non-trainable params: 0 (0.00 B)

Loss graphico con batch size = 32:



Loss graphico con batch_size = 1:



1. ¿Bajó más el error al añadir dos capas, o se estancó / empeoró? ¿Igual en NumPy y en Keras?

No, agregar dos capas no mejoró el resultado final. En la implementación a mano, la red $4 \times 3 \times 3 \times 3 \times 3$ terminó con un error de 0.13 después de 500 épocas, casi tres veces más alto que el de la red $4 \times 3 \times 3$ con 0.05. La curva de la red profunda baja en tres mesetas. Se explica porque en primero se queda cerca de 0.67, que es el error de una red que predice 1/3 para cada clase, y después baja por escalones.

En Keras con un batch de uno pasó algo similar porque la red profunda terminó con una pérdida 0.11 y la red normal con una de 0.06. Lo que cambia también es el inicio, porque en Keras la pérdida bajó más rápido desde las primeras épocas.

2. ¿Las curvas de la notebook 01 y de Keras se parecen con la misma topología? Si no, ¿qué diferencias de implementación podrían explicarlo?

La implementación a mano y Keras no se comportan igual. La primera diferencia es el tamaño del batch. En el código original no se especificaba `batch_size` en `model.fit`, así que Keras usó el valor por defecto, 32. Con solo 150 muestras, eso son 5 actualizaciones de pesos por época (unas 2 500 en 500 épocas), mientras que la implementación a mano actualiza los pesos con cada muestra: 150 por época (75 000 en total). Con un aprendizaje tan bajo (0.03), esas pocas actualizaciones no bastan para que la red aprenda bien (cerca de 0.222, que es lo que vale una red que predice 1/3 para cada clase).

Al usar `batch_size=1`, Keras hace las mismas 150 actualizaciones por época que la versión a mano y la pérdida final bajó a 0.075 (0.225 en la escala de la implementación a mano). Las curvas se parecen mucho más y Keras aprende bastante mejor.

Aun así, las curvas siguen siendo distintas. A mano se distinguen claramente tres etapas, mientras que en Keras se ven menos. Una posible explicación es el orden de los datos: Iris viene ordenado por clase y la versión a mano los recorre siempre en el mismo orden, mientras que Keras los mezcla en cada época (`shuffle=True` por defecto).

Otra diferencia es el cálculo del error. `calculate_error()` suma los cuadrados de las 3 salidas para cada muestra y luego promedia sobre las muestras. `MeanSquaredError` de Keras promedia también sobre las 3 salidas, así que su valor es un tercio del de la implementación a mano (una red que predice 1/3 para cada clase vale 0.667 a mano y 0.222 en Keras). Además, la loss de Keras es un promedio calculado durante la época, con los pesos cambiando, mientras que `calculate_error()` se evalúa al final de cada época. Por eso las dos curvas no están en la misma escala ni se miden en el mismo momento, y para compararlas hay que multiplicar la pérdida de Keras por 3.

3. Con sigmoides apiladas y MSE, ¿tiene sentido que una red más profunda no aprenda mejor en Iris? Relaciónalo con lo que viste en las gráficas.

Sí tiene sentido, por dos razones.

Primero, Iris es un problema pequeño (150 ejemplos, 4 atributos) y casi linealmente separable: la setosa se separa de las otras dos con una recta, y la dificultad real está en versicolor contra virginica. Una red $4 \times 3 \times 3$ ya tiene capacidad suficiente para eso, así que las capas extra no agregan capacidad útil.

Segundo, el gradiente se desvanece. La derivada de la sigmoide, $\sigma(z)(1-\sigma(z))$, vale como máximo 0.25, y en el backprop se multiplica una vez por cada capa. Con cuatro capas, los δ de las primeras capas son muy chicos y esas capas casi no aprenden. Eso se ve en mi gráfica: la curva se queda en mesetas (0.67 en la primera, 0.5 en la segunda) y baja solo cuando la red logra salir de cada una. Al llegar a las 500 épocas, la red profunda todavía estaba bajando en 0.16, mientras que la red más corta ya había llegado a 0.05.

En resumen, más profundidad no es mejor por sí sola: con sigmoides apiladas, MSE, $\eta = 0.03$ y solo 500 épocas, la red profunda aprende más lento y termina con un error mayor que la red simple.

Prueba:

Como prueba, puede verificar que las curvas del reporte son idénticas, a las de mis notebooks.